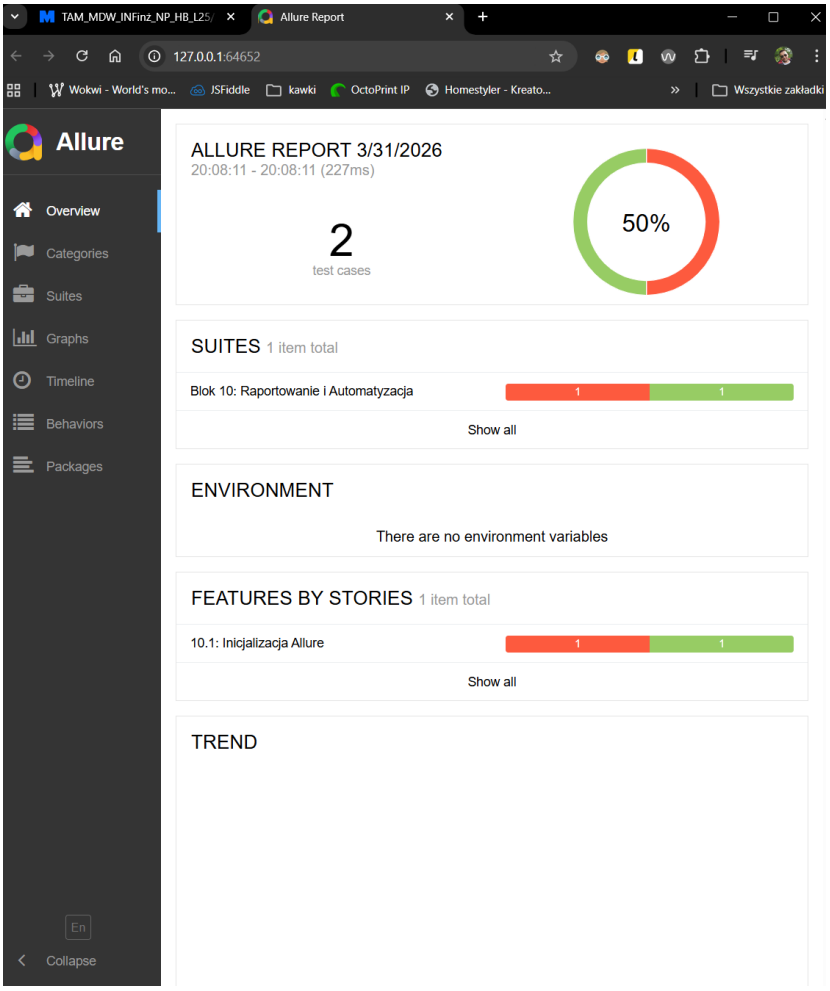
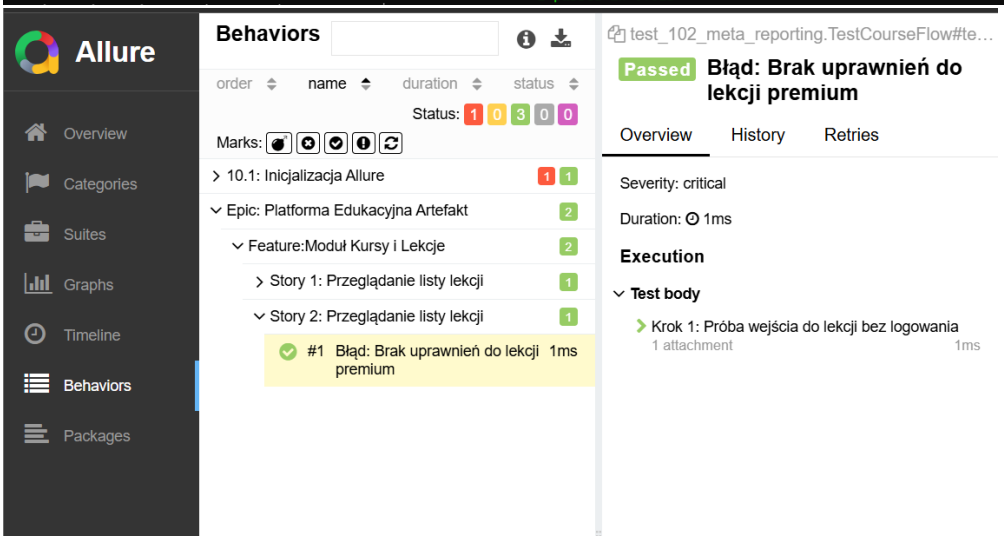
10.1

```
PS C:\Users\tomas\Desktop\TM-99080\Artefakt10> python -m pytest test_101_allure_init.py --alluredir=allure-results
========================================================= test session starts =========================================================
platform win32 -- Python 3.12.1, pytest-8.3.5, pluggy-1.5.0
rootdir: C:\Users\tomas\Desktop\TM-99080\Artefakt10
plugins: allure-pytest-2.15.3
collected 2 items

test_101_allure_init.py .F                                                                                                          [100%]

============================================================= FAILURES ==============================================================
_____________________________________________ TestAllureSetup.test_intentional_failure _____________________________________________

self = <test_101_allure_init.TestAllureSetup object at 0x0000016C1D0128D0>

    @allure.title("Test Przykładowego Błędu")
    def test_intentional_failure(self):
        """Ten test ma celowo pokazać, jak Allure raportuje błędy."""
        with allure.step("Krok 1: Porównaj dane z API"):
            expected = "Tytuł z API"
            actual = "Błędny tytuł"
>           assert expected == actual, f"Oczekiwano {expected}, a otrzymano {actual}"
E           AssertionError: Oczekiwano Tytuł z API, a otrzymano Błędny tytuł
E           assert 'Tytuł z API' == 'Błędny tytuł'
E
E             - Błędny tytuł
E             + Tytuł z API

test_101_allure_init.py:29: AssertionError
======================================================= short test summary info =======================================================
FAILED test_101_allure_init.py::TestAllureSetup::test_intentional_failure - AssertionError: Oczekiwano Tytuł z API, a otrzymano Błęd
ny tytuł
================================================== 1 failed, 1 passed in 0.45s ================================================
```



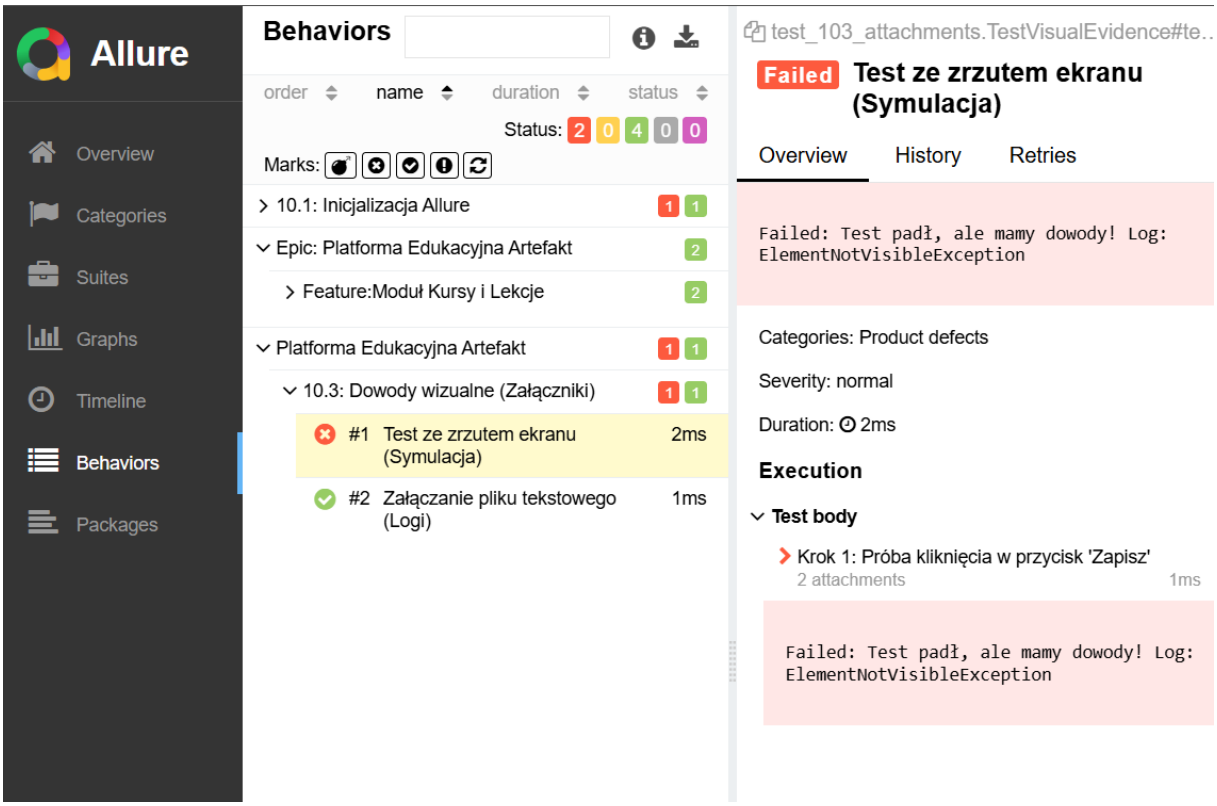
10.2

```
PS C:\Users\tomas\Desktop\TM-99080\Artefakt10> python -m pytest .\test_102_meta_reporting.py --alluredir=allure-results
============================================================= test session starts ==============================================================
platform win32 -- Python 3.12.1, pytest-8.3.5, pluggy-1.5.0
rootdir: C:\Users\tomas\Desktop\TM-99080\Artefakt10
plugins: allure-pytest-2.15.3
collected 2 items

test_102_meta_reporting.py ..                                                                                                                [100%]

============================================================== 2 passed in 0.02s ===============================================================
```



10.3

```
plugins: allure-pytest-2.15.3
collected 2 items

test_103_attachments.py F.                                                                                                                   [100%]

================================================================= FAILURES =================================================================
_____________________________________________ TestVisualEvidence.test_screenshot_on_fail _____________________________________________

self = <test_103_attachments.TestVisualEvidence object at 0x0000023422B37650>

    @allure.title("Test ze zrzutem ekranu (Symulacja)")
    def test_screenshot_on_fail(self):
        with allure.step("Krok 1: Próba kliknięcia w przycisk 'Zapisz'"):
            # Symulujemy błąd znalezienia elementu
            try:
                element_found = False
                if not element_found:
>                   raise Exception("ElementNotVisibleException")
E                   Exception: ElementNotVisibleException

test_103_attachments.py:16: Exception

During handling of the above exception, another exception occurred:

self = <test_103_attachments.TestVisualEvidence object at 0x0000023422B37650>

    @allure.title("Test ze zrzutem ekranu (Symulacja)")
    def test_screenshot_on_fail(self):
        with allure.step("Krok 1: Próba kliknięcia w przycisk 'Zapisz'"):
            # Symulujemy błąd znalezienia elementu
            try:
                element_found = False
                if not element_found:
                    raise Exception("ElementNotVisibleException")
            except Exception as e:
                # TUTAJ DZIEJE SIĘ MAGIA:
                # W rzeczywistym teście byłoby: driver.get_screenshot_as_png()
                allure.attach(
                    "Tutaj byłby obraz z telefonu",
                    name="Screenshot_Error_01",
                    attachment_type=AttachmentType.TEXT # Zmienimy na PNG w pełnym Appium
                )
                allure.attach(
                    "JSON Error Data: {'code': 404, 'msg': 'Not Found'}",
                    name="API_Response",
                    attachment_type=AttachmentType.JSON
                )
>               pytest.fail(f"Test padł, ale mamy dowody! Log: {e}")
E               Failed: Test padł, ale mamy dowody! Log: ElementNotVisibleException

test_103_attachments.py:30: Failed
========================================================= short test summary info ==========================================================
FAILED test_103_attachments.py::TestVisualEvidence::test_screenshot_on_fail - Failed: Test padł, ale mamy dowody! Log: ElementNotVisibleExc
eption
=================================================== 1 failed, 1 passed in 0.04s ====================================================
PS C:\Users\tomas\Desktop\TM-99080\Artefakt10>
```



10.4

```
test_103_attachments.py:16: Exception

During handling of the above exception, another exception occurred:

self = <test_103_attachments.TestVisualEvidence object at 0x000001E59EBEB080>

    @allure.title("Test ze zrzutem ekranu (Symulacja)")
    def test_screenshot_on_fail(self):
        with allure.step("Krok 1: Próba kliknięcia w przycisk 'Zapisz'"):
            # Symulujemy błąd znalezienia elementu
            try:
                element_found = False
                if not element_found:
                    raise Exception("ElementNotVisibleException")
            except Exception as e:
                # TUTAJ DZIEJE SIĘ MAGIA:
                # W rzeczywistym teście byłoby: driver.get_screenshot_as_png()
                allure.attach(
                    "Tutaj byłby obraz z telefonu",
                    name="Screenshot_Error_01",
                    attachment_type=AttachmentType.TEXT # Zmienimy na PNG w pełnym Appium
                )
                allure.attach(
                    "JSON Error Data: {'code': 404, 'msg': 'Not Found'}",
                    name="API_Response",
                    attachment_type=AttachmentType.JSON
                )
>               pytest.fail(f"Test padł, ale mamy dowody! Log: {e}")
E               Failed: Test padł, ale mamy dowody! Log: ElementNotVisibleException

test_103_attachments.py:30: Failed
=================================================== short test summary info ===================================================
FAILED test_101_allure_init.py::TestAllureSetup::test_intentional_failure - AssertionError: Oczekiwano Tytuł z API, a otrzymano Błędny tytu
ł
FAILED test_103_attachments.py::TestVisualEvidence::test_screenshot_on_fail - Failed: Test padł, ale mamy dowody! Log: ElementNotVisibleExc
eption
================================================= 2 failed, 4 passed in 0.40s =================================================

📝 KROK 3: Dokumentowanie przebiegu Pipeline...

📊 KROK 4: Generowanie raportu Allure...
Report successfully generated to allure-report

🧹 KROK 5: Zamykanie i usuwanie kontenerów...
[+] down 2/2
 ✔Container appium-server           Removed                                                                                  1.2s
 ✔Network artefakt09_mobile-testing Removed                                                                                  0.2s

============================================================
🏆 PIPELINE ZAKOŃCZONY POPRAWNIE!
📍 Raport wygenerowany: C:\Users\tomas\Desktop\TM-99080\Artefakt10/allure-report/index.html
💡 Użyj 'allure serve allure-results' aby go wyświetlić.
============================================================

PS C:\Users\tomas\Desktop\TM-99080\Artefakt10>
```

Allure

- Overview
- Categories
- Suites
- Graphs
- Timeline
- Behaviors
- Packages

En

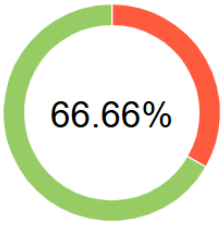
Collapse

ALLURE REPORT 3/31/2026

20:19:21 - 20:19:21 (290ms)

6

test cases



SUITES 3 items total

Blok 10: Raportowanie i Automatyzacja	1 / 1
test_103_attachments	1 / 1
test_102_meta_reporting	2

Show all

ENVIRONMENT

Pipeline.Status	SUCCESS
Execution.Time	2026-03-31 20:19:21
Execution.Type	Automated Master Script
Infrastructure	Docker (Appium Server)
Platform	win32

Show all

FEATURES BY STORIES 3 items total

10.1: Inicjalizacja Allure	1 / 1
Platforma Edukacyjna Artefakt	1
Epic: Platforma Edukacyjna Artefakt	1

10.5

Preview README.md

🧮 Mobile Automation & Cloud-Ready Testing Suite

Prowadzący: mgr Mariusz Dworniczak **Student:** Tomasz Bednarz **Numer Albumu:** 99080

🏗️ Architektura Projektu (Marketing & Tech Stack)

Ten projekt to kompletny ekosystem testowy oparty na podejsciu **Cloud-Ready / Headless**. Zamiast polegać na ciężkich emulatorach, skupiamy się na narzędziach CLI, analizie statycznej, konteneryzacji (Docker) oraz automatyzacji procesów (Pipeline).

Główne technologie:

- **Język:** Python 3.10+
- **Automatyzacja UI:** Appium 2.x (Mobile Engine)
- **Infrastruktura:** Docker & Docker Compose
- **Raportowanie:** Allure Framework
- **Analiza:** MobSF (Static Analysis) & ADB CLI

🗓️ PRZEBIEG LABORATORIUM (Kamienie Milowe)

🔹 BLOK 1: Tooling & Environment (Infrastruktura)

Przygotowanie bazy narzędziowej w modelu kontenerowym.

- **Co zrobiono:** Pobranie i konfiguracja obrazów `appium`, `android-sdk` oraz `mobsf`.
- **Wniosek:** Zastosowanie kontenerów Docker umożliwia szybkie i powtarzalne przygotowanie środowiska testowego bez konfliktów zależności. Dzięki temu unikamy problemów związanych z konfiguracją lokalną, a cały stack (Appium, Android SDK, MobSF) działa identycznie na każdym stanowisku, co zwiększa stabilność i przenośność projektu.

🔹 BLOK 2: Debugowanie i Analiza Statyczna (MobSF)

Zrozumienie "wnętrza" aplikacji mobilnej przed przystąpieniem do testów.

- **Co zrobiono:** Wykorzystanie MobSF do skanowania plików APK pod kątem podatności i uprawnień.
- **Wniosek:** Analiza statyczna APK pozwala zidentyfikować potencjalne podatności, nadmiarowe uprawnienia oraz niebezpieczne fragmenty kodu jeszcze przed uruchomieniem aplikacji. Dzięki temu tester może szybciej zrozumieć strukturę aplikacji i skupić się na najbardziej ryzykownych obszarach podczas testów dynamicznych.

🔹 BLOK 3-4: Fundamenty Skryptowania (Python for QA)

Budowa logiki testowej w języku Python.

- **Co zrobiono:** Nauka podstawowych struktur danych w Pythonie, takich jak listy, słowniki i krotki, oraz definiowania funkcji i obsługi warunków (if/else). Wykorzystanie pętli (for, while) do iteracji oraz modularnego podejścia do kodu